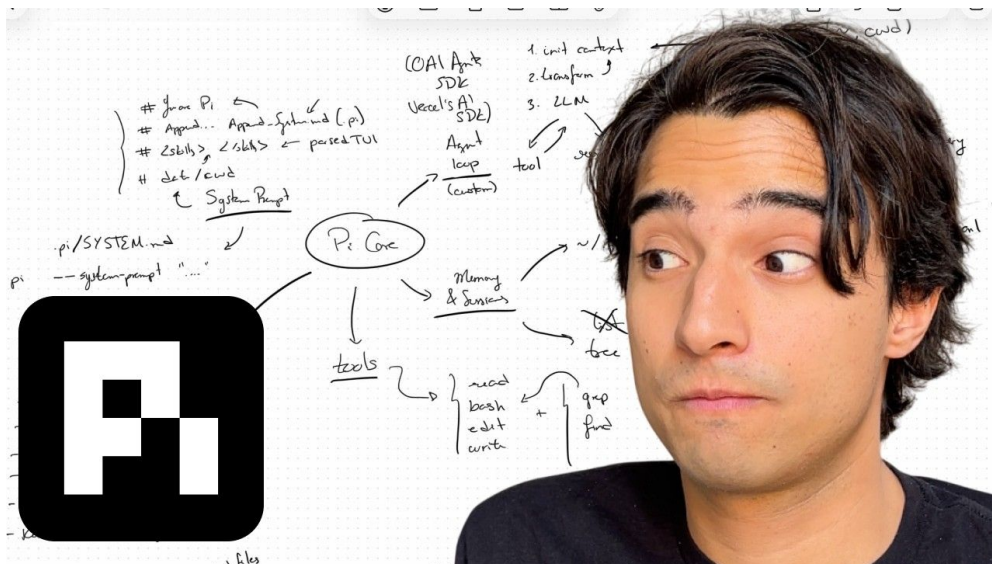


PI 架构深度解析：Agent 循环、工具调用、TUI 与扩展机制

63 号炼金工坊 & Codex

2026-06-06



视频作者：63 号炼金工坊

时长：38:58

B 站链接：<https://www.bilibili.com/video/BV1J87Q6xEL7>

参考文章：<https://alejandro-ao.com/pi-architecture/>

YouTube: How Pi Works: Agent Architecture, Tools, TUI, and Skills

目录

1 Pi 项目概述	3
1.1 什么是 Pi	3
1.2 整体架构概览	3
1.3 Pi 的核心概念清单	3
2 Agent Core: 核心 Agent 循环	4
2.1 核心循环流程	4
2.1.1 详细的每一步	4
2.2 上下文构建 (Context Initialization)	5
2.3 会话与对话状态 (Session)	5
3 工具系统 (Tool System)	6
3.1 工具的桥梁作用	6
3.2 典型工具列表	6
3.3 工具 Schema	6
3.4 工具执行流程	7
4 分层提示词系统	7
4.1 五层提示词体系	7
4.2 各层的作用	7
5 扩展机制 (Extensions)	8
5.1 什么是 Extension	8
5.2 Extension 的加载时机	8
6 Pi Interactive: 终端 UI 层	9
6.1 TUI 的职责	9
6.2 双层的交互流程	9
7 上下文压缩 (Compaction)	10
7.1 为什么需要 Compaction	10
7.2 Compaction 保留的信息	10
7.3 Compaction vs 简单截断	10
8 技能系统 (Skills)	10
8.1 什么是 Skill	10
8.2 Skill 示例: 视频发布 workflow	11
8.3 Skills 与 Extensions 的区别	11
9 完整架构全景图	11
10 为什么这套架构有效	13
10.1 简洁抽象的力量	13
10.2 Pi 对 Agent 设计的启示	13
10.3 与其他编码 Agent 的对比	13

11 总结与延伸	14
11.1 视频核心回顾	14
11.2 拓展阅读与资源	14
11.3 进一步思考的问题	14

RPC 与 SDK:

1 Pi 项目概述

1.1 什么是 Pi

Pi 是一个轻量级的**终端编码助手** (Terminal Coding Assistant)，由 Alejandro AO 开发，完全开源。与 Claude Code、Codex CLI 等大型编码助手不同，Pi 的设计哲学是**极简 + 可理解**：

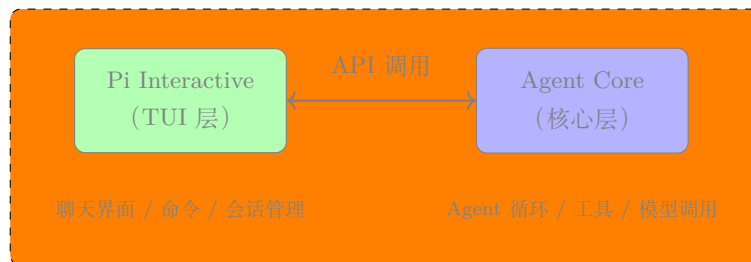
- 核心代码量小，架构清晰，适合学习。
- 不依赖复杂的框架，从零构建完整的 Agent 系统。
- 作为一个**教育项目**，Pi 展示了编码 Agent 应有的所有核心组件。

Pi 的设计哲学

Pi 不是一个黑盒产品，而是一个**可理解的架构参考**。它的代码量足够小，让你能理解每一行代码的作用；同时它包含了真实 Agent 所需的所有关键组件：Agent 循环、工具系统、会话管理、上下文压缩、TUI、扩展机制。

1.2 整体架构概览

Pi 由**两个独立的层**组成，这种分离是其架构的核心设计决策：



为什么要分层？

- **关注点分离**：核心层只关心 Agent 循环和工具执行，不关心用户界面。
- **可复用性**：同一个 Agent Core 可以被 TUI、CLI、SDK 集成、HTTP API 等多种方式调用。
- **可测试性**：核心层可以独立测试，不需要模拟终端界面。

1.3 Pi 的核心概念清单

概念	说明
Agent Core	核心 Agent 循环引擎，负责上下文构建 → 模型调用 → 响应解析 → 工具执行
TUI (Pi Interactive)	终端用户界面，负责聊天交互、命令处理、会话管理
Context Init	每次模型调用前构建上下文的流程
Tool System	让模型能够通过函数调用与操作系统交互的机制
Session	跨轮次的对话状态持久化

有 OpenAI Agent SDK, Vercel AI SDK

~/.pi/agent/sessions { 目录列表 { ID (jsonl) { ID parent role message ... }

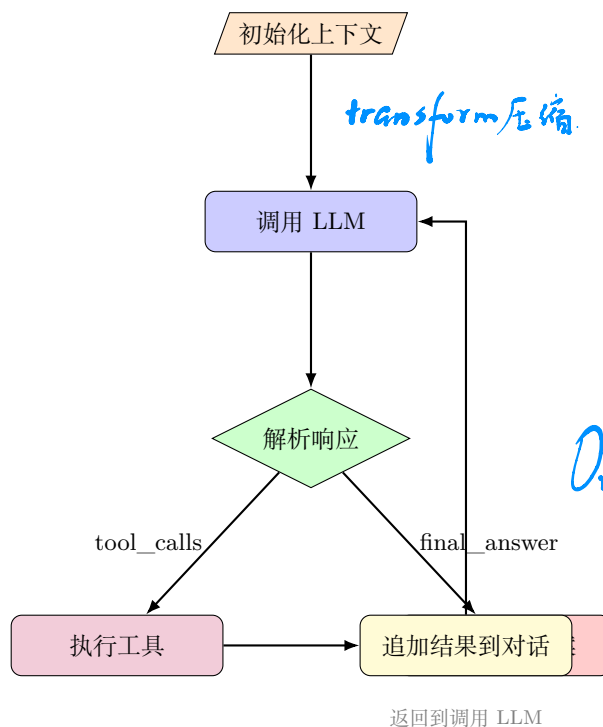
/ tree

概念	说明
System Prompt	分层的提示词体系（基础/项目/技能/用户）
Extension	插件式扩展机制，可添加工具、命令、自定义行为
Compaction	上下文压缩机制，突破长度限制
Skill	可复用的 workflow 指令，替代人肉记忆复杂流程

2 Agent Core: 核心 Agent 循环

2.1 核心循环流程

Agent Core 的核心是一个循环（Loop）。每次交互的执行流程完全一致：



transform 压缩

Or -- system-prompt "..."

append-system.md

再加

system prompt

2.1.1 详细的每一步

1. **初始化上下文 (Context Initialization)**: 构建发送给模型的消息列表，包括系统提示词、项目指令、可用工具描述、会话历史、用户消息、技能/扩展指令。
2. **调用 LLM**: 将构建好的上下文发送给语言模型，等待响应。
3. **解析响应 (Parse Response)**: 判断模型的输出是最终答案 (Final Answer) 还是工具调用 (Tool Calls)。
4. **执行工具 (Execute Tools)**: 如果是工具调用，执行对应的工具函数，获取结果。
5. **追加结果**: 将工具执行结果追加到对话历史中，然后返回第 2 步继续循环。
6. **返回答案**: 如果是最终答案，返回给用户，结束本轮交互。

agent s.md

Pi 循环的核心洞察

”有趣的部分不在于有一个循环，而在于在调用模型之前放入了什么到上下文中，以及工具结果如何被反馈到对话中。”——Alejandro AO

2.2 上下文构建 (Context Initialization)

当一个 Session 开始时，Pi 构建的上下文包含以下层次：



Pi 的 Base Prompt

Pi 的基础系统提示词故意保持**小巧** (small)，这让它的行为容易理解和定制。这也是 Pi 成为优秀教育项目的原因之一——架构足够紧凑可以完整理解，但包含了真实编码 Agent 所需的所有核心组件。

2.3 会话与对话状态 (Session)

Session 在多次交互之间保持连续性，它存储：

- **对话历史** (Conversation History)：完整的消息列表。
- **工具执行结果**：每次工具调用的输入和输出。
- **必要状态**：继续任务所需的所有信息。

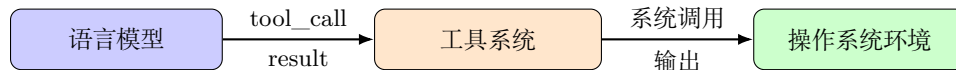
没有 Session 会怎样？

如果没有 Session，每一次交互都是孤立的。Agent 无法记住之前读了什么文件、做了什么修改、遇到了什么错误。有了 Session，Pi 才能像一个持续协作的伙伴，支持多步骤的编码任务（读文件 → 改代码 → 运行 → 检查错误 → 迭代）。

3 工具系统 (Tool System)

3.1 工具的桥梁作用

工具是模型与环境之间的桥梁。语言模型本身无法直接读取文件、执行命令或编辑代码——工具赋予了它这些能力。



3.2 典型工具列表

Pi 作为编码助手，提供了以下核心工具：

工具	类型	功能
read_file	文件操作	读取指定文件内容到上下文
write_file	文件操作	写入或覆盖指定文件
edit_file / patch	文件操作	精确的查找替换编辑
run_command	Shell 执行	运行任意 Shell 命令
search_files / grep	搜索	在代码库中搜索内容和文件名
list_directory	文件系统	列出目录内容

Handwritten notes in blue ink: A bracket groups 'read', 'bash', and 'edit' on the left, with 'write' below it. An arrow points from this group to another bracket on the right that groups 'grep' and 'find', with '(只读)' (read-only) next to it.

3.3 工具 Schema

每个工具在系统提示词中都有一段描述 (Schema)，告诉模型：

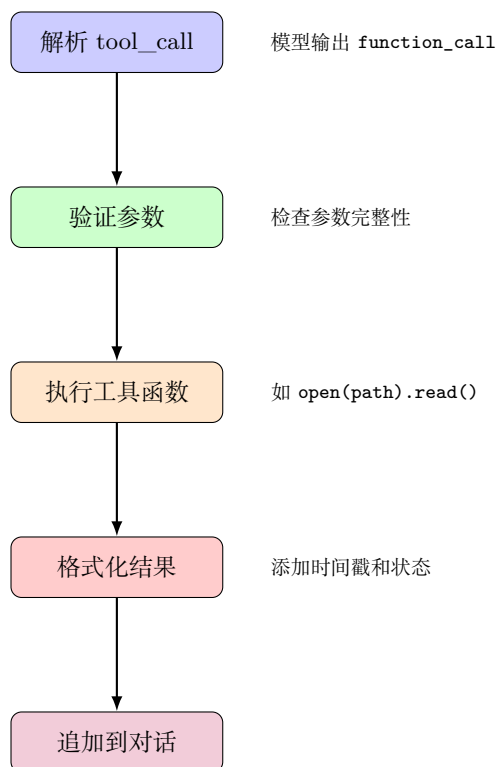
工具 Schema 示例

工具名：read_file 描述：读取指定路径的文件内容输入：path（必填）- 文件路径 offset（可选）- 起始行号 limit（可选）- 最大行数输出：文件内容字符串

工具描述的关键性

模型根据这些描述决定何时调用什么工具以及传入什么参数。描述越精确，模型的行为越可靠。这是 Prompt Engineering 在 Agent 系统中的核心应用。

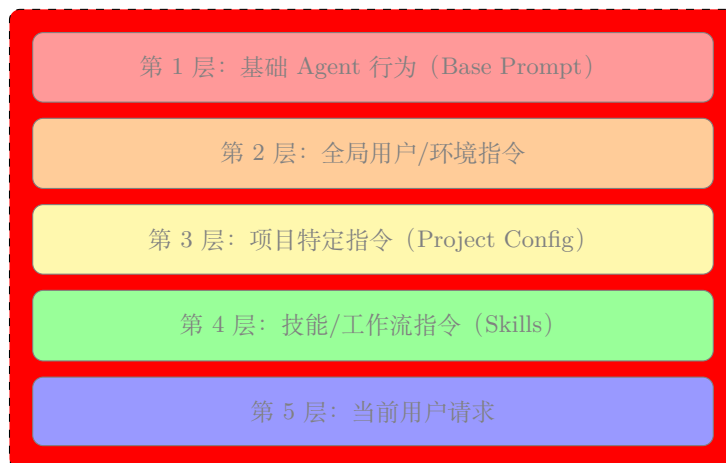
3.4 工具执行流程



4 分层提示词系统

4.1 五层提示词体系

Pi 的提示词系统采用分层叠加的方式，每一层在前一层的基础上增加：



4.2 各层的作用

1. **Base Prompt**: Pi 的核心人格和基础行为规则。保持小巧，便于理解。
2. **Global Instructions**: 用户级别的全局配置，如默认使用的工具链 (Python、Node.js 等)。
3. **Project Instructions**: 项目级别的指令，如”这个项目使用 uv 管理依赖”、”测试用 pytest”。
4. **Skills**: 可复用的工作流指令，如视频发布流程、部署流程等。

5. **User Request**: 当前轮次用户的具体问题或任务。

分层设计的价值

这种分层让 Pi 在保持通用性的同时，可以灵活适配不同的代码库。项目 A 可能用 `uv` 管理 Python，项目 B 可能用 `make`——通过 Project Instructions 层，Pi 自动调整行为而不需要修改基础代码。

5 扩展机制 (Extensions)

用 JS 编写

5.1 什么是 Extension

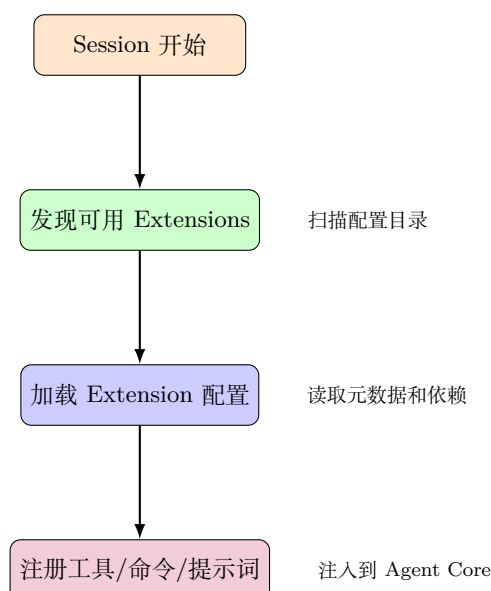
Extension 是 Pi 的**插件系统**，允许在不修改核心代码的前提下添加功能。Extension 可以贡献：

- **额外的工具** (Additional Tools): 如数据库查询工具、API 调用工具。
- **自定义命令** (Custom Commands): 如 `/deploy`、`/test`。
- **修改提示词** (Modified Prompts): 向系统提示词追加内容。
- **项目特定集成** (Project-Specific Integrations): 按需加载。
- **额外的工作流行为**。

架构原则

”保持核心循环简洁，然后为所有可选的或可定制的内容创建扩展点。” ——Keep the core loop small, then create extension points for everything optional.

5.2 Extension 的加载时机



6 Pi Interactive: 终端 UI 层

6.1 TUI 的职责

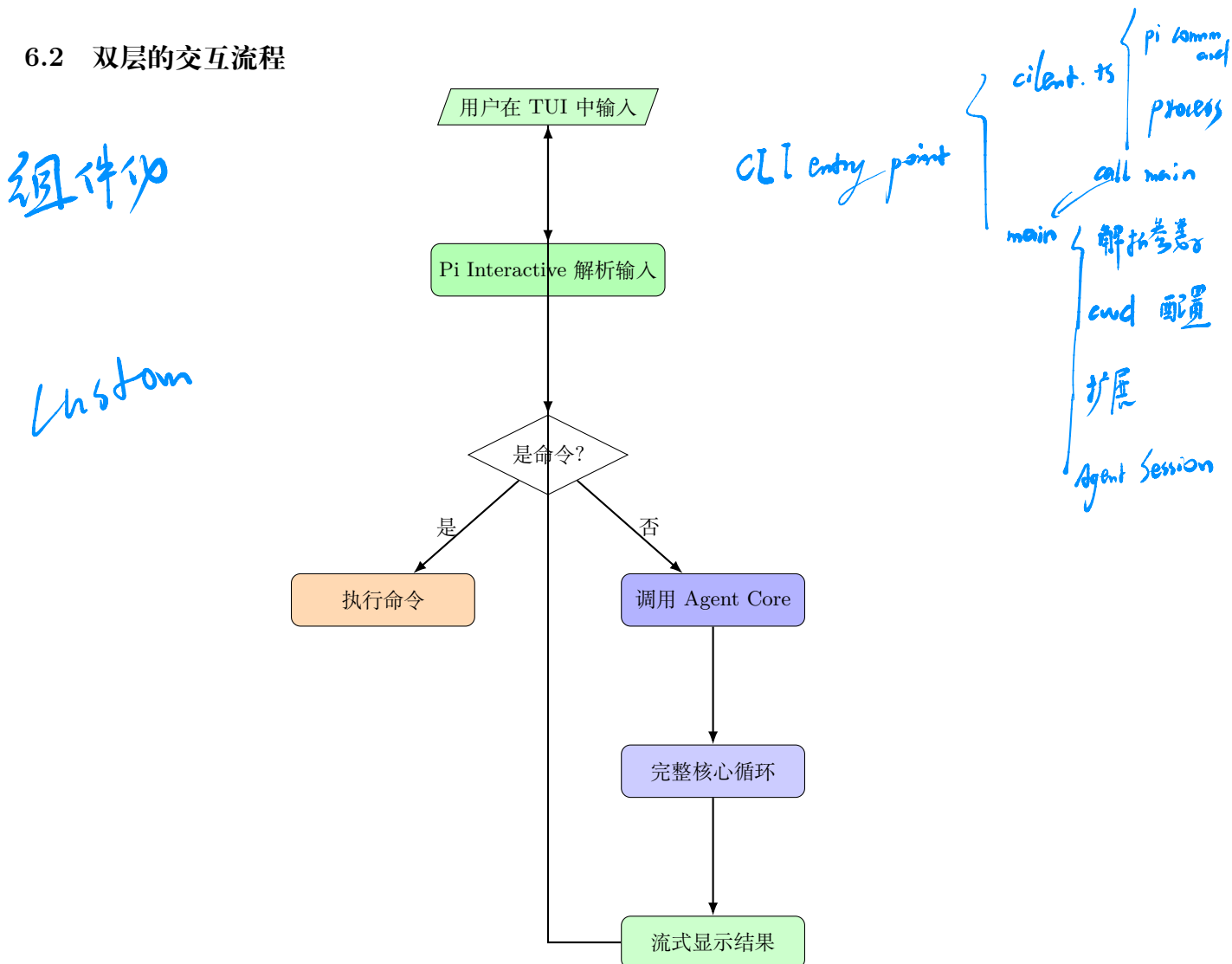
Pi Interactive 是终端用户界面层，它为 Agent Core 提供了一个可交互的包装器。它的职责包括：

功能	说明
聊天界面	输入框、流式输出、Markdown 渲染
命令系统	/help、/sessions、/compact 等命令
会话管理	创建、切换、删除、重命名会话
压缩控制	手动或自动触发上下文压缩
技能接口	加载、查看、应用技能
状态显示	Token 用量、延迟、当前会话信息

核心区分

TUI 不是 Agent 本身——它是围绕 Agent Core 的一个用户界面。这种分离确保 Agent 运行时（Core）专注于循环和逻辑，TUI 专注于用户体验。

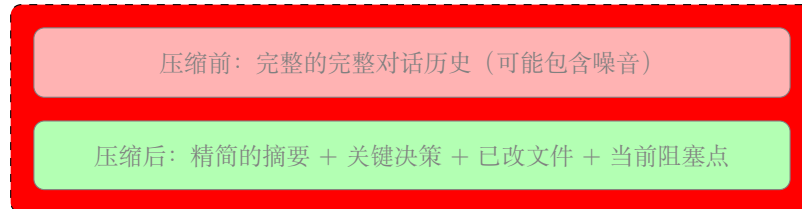
6.2 双层的交互流程



7 上下文压缩 (Compaction)

7.1 为什么需要 Compaction

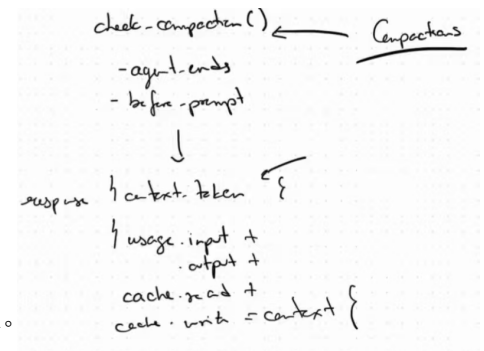
长时间的会话会面临 LLM 上下文窗口的限制。随着对话增长，早期的重要信息可能会被截断。Compaction 的作用是压缩或总结较早的对话状态，保留关键信息的同时保持在窗口内。



7.2 Compaction 保留的信息

好的 Compaction 会保存:

- 用户的目标 (User's Goal): 最初要解决的问题。
- 重要决策 (Important Decisions): 架构选择、技术方案确定。
- 已修改的文件 (Files That Were Changed): 改了什么、为什么改。
- 当前的阻塞点 (Current Blockers): 卡在什么问题上了。
- 相关的命令和结果 (Relevant Commands and Results): 关键执行输出。



7.3 Compaction vs 简单截断

维度	Compaction	简单截断 (Truncation)
信息保留	选择性保留关键信息	直接丢掉最早的消息
语义完整性	LLM 总结, 保持连贯	可能中断正在进行的任务
Token 效率	高 (压缩比通常 5-10x)	低 (只是删除了前缀)
实现复杂度	中等 (需要额外 LLM 调用)	极低

8 技能系统 (Skills)

8.1 什么是 Skill

Skill 是可复用的工作流指令——它不是给人看的文档，而是给 Agent 看的操作指令。

Skill 的本质

Skill = 一段结构的提示词文本，描述了如何完成一个特定的多步骤工作流。Agent 在加载 Skill 后，将其内容注入到系统提示词中，从而在执行相关任务时遵循预设的流程。

注意: skill 本身不会进入 agent core.

8.2 Skill 示例：视频发布 workflow

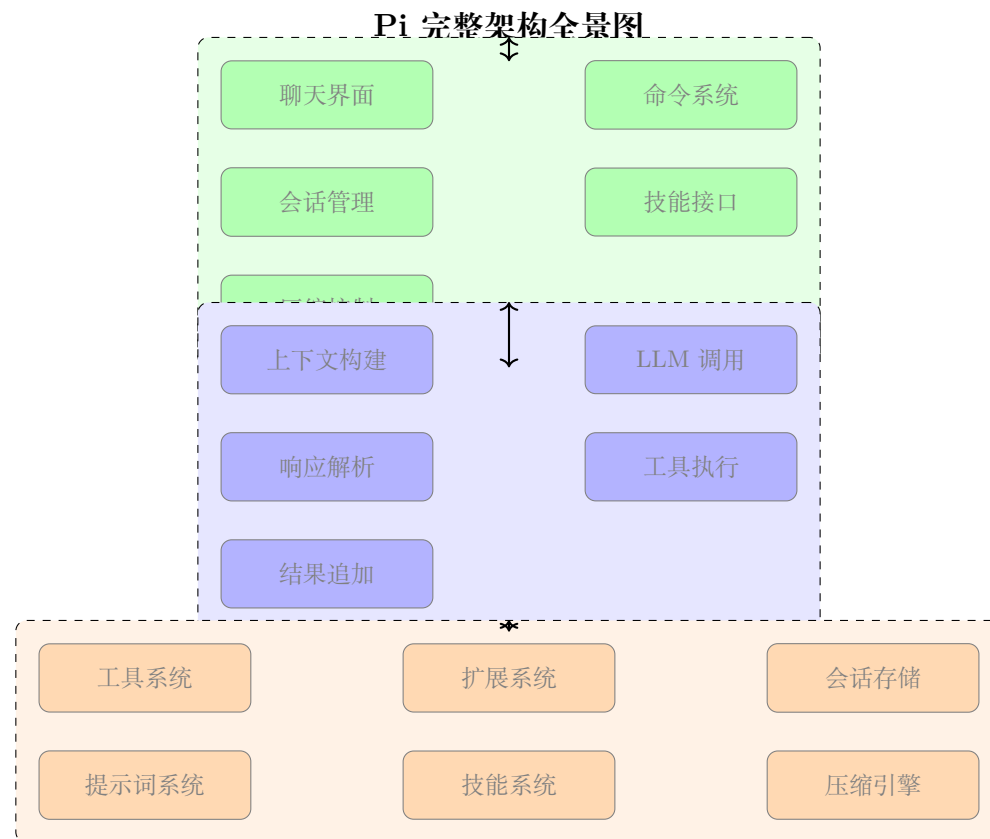
Skill 示例：视频发布流程

视频发布 workflow 1. 制作原始剪辑 2. 使用工具生成字幕 (Caption) 3. 添加叠加元素 (Overlays) 4. 渲染最终视频 5. 上传到发布平台 6. 撰写博客文章 7. 创建社交媒体帖子 8. 在各平台同步发布

8.3 Skills 与 Extensions 的区别

维度	Skills	Extensions
本质	提示词文本	可执行代码
影响范围	修改 Agent 的指令和行为	添加工具、命令、集成
是否需要编程	不需要 (纯文本)	需要 (代码实现)
共享方式	文件共享/Skill 市场	包管理器/插件市场
复杂度	低	中到高

9 完整架构全景图



10 为什么这套架构有效

10.1 简洁抽象的力量

Pi 建立在一小组清晰的抽象之上：

- **Core Loop**：与模型通信的核心循环。
- **Context Builder**：组装指令和历史。
- **Tool System**：让模型能够作用于外部环境。
- **Sessions**：持久化状态。
- **Extensions**：可选的扩展能力。
- **TUI**：可用的用户界面。
- **Compaction**：突破上下文限制。
- **Skills**：可复用的工作流。

Pi 的启示

”这些组件单独看没有一个特别复杂。但合在一起，它们构成了一个能够胜任实际编程任务的 Agent——同时代码量仍然小到足够让你理解它的每一部分。”——Alejandro AO

10.2 Pi 对 Agent 设计的启示

1. **分层架构是基础**：核心层和 UI 层分离让系统更灵活。
2. **提示词系统需要分层**：Base + Global + Project + Skill 的叠加模式。
3. **工具是 Agent 能力的来源**：工具系统设计的好坏决定了 Agent 的边界。
4. **扩展性来自插件化**：Extension 机制让核心保持简洁。
5. **Compaction 是长会话的必需品**：没有它，Agent 无法处理复杂任务。

10.3 与其他编码 Agent 的对比

维度	Pi	Claude Code	Codex CLI	Hermes
开源	✓	部分	✓	✓
架构复杂度	低	高	中	中
TUI	Textual	内建	内建	无 (CLI only)
Skill 系统	✓	✓	AGENTS.md	✓
Extension	✓	插件 + MCP	插件 + MCP	MCP + 内建
上下文压缩	✓	Headroom 类似	compact API	Headroom
代码量	小	大	中	中

11 总结与延伸

11.1 视频核心回顾

1. **Pi 的分层架构**: Agent Core (循环/工具) 和 TUI (界面/管理) 完全分离。
2. **核心循环**: 上下文构建 → LLM 调用 → 解析 → 工具执行 → 循环。
3. **工具系统**: 模型通过 Tool Schema 了解和使用外部工具。
4. **分层提示词**: 5 层体系, 从基础行为到用户请求逐层叠加。
5. **Extensions**: 插件式扩展, 不修改核心即可添加功能。
6. **Compaction**: 用 LLM 做智能上下文压缩。
7. **Skills**: 可复用的工作流指令。
8. **教育价值**: Pi 是学习 Agent 架构的最佳参考实现之一。

11.2 拓展阅读与资源

- **Pi GitHub 仓库**: <https://github.com/torantulino/pi>
- **Pi 架构文章**: How Pi Works: Agent Architecture, Tools, TUI, and Skills
- **YouTube 视频**: Pi Architecture Deep Dive
- **Claude Code 架构**: Anthropic 官方文档
- **Codex CLI**: GitHub
- **Hermes Agent**: 官方文档

11.3 进一步思考的问题

- 如何将 Pi 的 Extension 机制与 MCP (Model Context Protocol) 集成?
- Pi 的 Compaction 策略如何处理工具执行结果? 不同模型对压缩质量的差异?
- Skills 系统是否可以从纯文本升级为程序化的工作流 DAG?
- 对于生产环境, Pi 的 Core 层需要增加哪些安全性 (沙箱、权限控制)?